

RBE549 HW0: Alohomora!

Samruddhi Naukudkar
Robotics Engineering Department
Worcester Polytechnic Institute
Worcester, 01609
Email: snaukudkar@wpi.edu

Abstract—This homework has two phases: 1) Shake my boundary, and 2) Deep dive into deep study. The first step employs the Pb (probability of boundary) lite method for boundary detection. The feature vector is quantized using various filterbanks and Kmeans. The second phase includes a basic deep learning model, as well as three well-known models: ResNet, ResNeXt, and DenseNet. .

I. PHASE 1: SHAKE MY BOUNDRY

Boundary detection is a significant difficulty in computer vision since it requires determining where one object finishes and another begins. While it appears straightforward, finding boundaries in a single image is tough because it frequently requires detailed knowledge of the items in the image. One fundamental method for detecting borders is to look for sharp variations in brightness, which are called edges.

Traditional edge detection approaches, such as the Canny and Sobel algorithms, concentrate on detecting these brightness variations. A recent method, known as the pb (probability of boundary) algorithm, performs better by taking into account changes in texture and color as well as brightness. This enables it to eliminate false positives, which traditional approaches frequently produce in areas with a lot of texture.

1.Filtering: Preparing an image for analysis.

2.K-Means Clustering: which involves creating texture brightness, and color maps.

3.Chi-square Distance using Half Disks: Calculating gradients from maps.

4.Merging maps: Merging texture, brightness, and color maps utilizing Canny and Sobel techniques.

This method uses many features in a picture to improve accuracy and surpass classic edge detection techniques.

A. Creation of Filter Banks:

1) Derivative of Gaussian (DoG) filters: A simple yet effective filter bank consists of orientated Derivative of Gaussian (DoG) filters. These filters can be made by convolving a simple Sobel filter with a Gaussian kernel and rotating the result. Figure 2 displays the filter bank generated with two scales and 16 orientations.

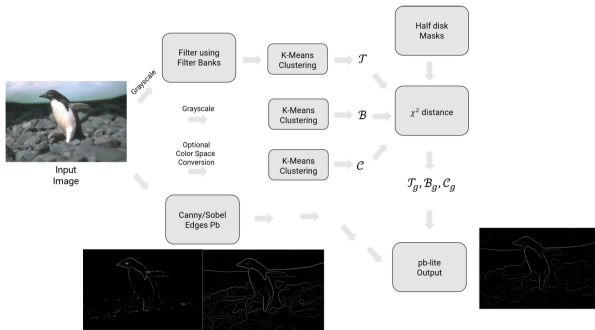


Fig. 1: Overview of the Pb-lite Algorithm.

To improve on these ideas, a simplified version of the Pb-lite algorithm was created, whose algorithm overview is shown in the Figure 1 above. It outperforms prior approaches by combining texture, brightness, and color information to detect boundaries. The procedure consists of four basic steps:

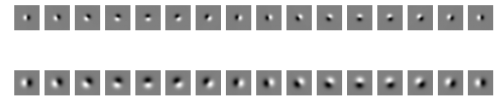
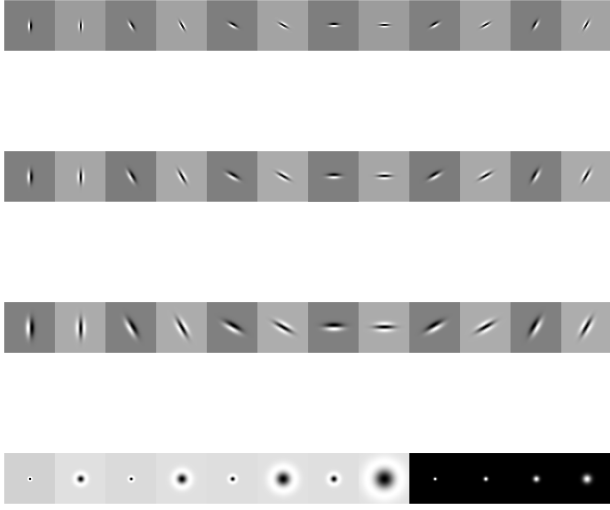


Fig. 2: DoG Filters.

2) Leung-Malik (LM) filters: The Leung-Malik (LM) filter bank is made up of 48 filters that are meant to extract features at many scales and orientations. This implementation contains two versions of the LM filter bank:

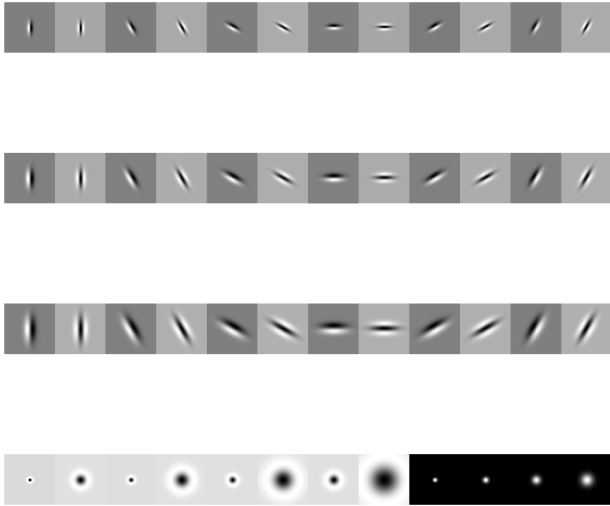
1) LM Small Filter Bank was created using scales: $(1, \sqrt{2}, 2, 2\sqrt{2})$

The LM Small filter bank is visualized in below Figure.



2) LM Large Filter Bank was created with scales: $(\sqrt{2}, 2, 2\sqrt{2}, 4)$

The LM Large filter bank is shown in below Figure.



3) **Gabor filters:** There are 40 filters in the Gabor filter bank. A Gabor filter is a Gaussian kernel function modulated by a sinusoidal plane wave, as stated on the course website. Figure 5 displays the filter bank created using five scales and eight orientations.



B. Texton, Brightness, Color Maps:

A vector of filter responses centered on each pixel is produced when an input image is filtered using each filter in a filter bank. For instance, every pixel will have N filter responses if the filter bank has N filters. It is possible to interpret the distribution of these N -dimensional filter responses as encoding the image's texture properties.

This approach is made simpler by substituting a discrete texton ID for each N -dimensional vector. This is accomplished by grouping the filter responses from every pixel into K textons using KMeans clustering. As a result, rather than using a high-dimensional real-valued vector to represent each pixel, a single cluster ID is used.

Variations in the image's brightness are captured by the brightness map. This entails utilizing KMeans clustering to group the brightness values of the image's grayscale equivalent. Although this can be changed for research, the brightness values are typically grouped into 16 clusters. The brightness map B is the name given to the clustered output.

Likewise, the color map records the image's chrominance information or color transitions. Here, KMeans clustering is used to group the color values (such as RGB values or values from other color spaces like YCbCr, HSV, or Lab). Once more, grouping into 16 groups is a popular option, but this can be changed as necessary. The color map C is the result of this clustering technique. All the maps are shown in the Figure [4]

C. Texton, Brightness, Color Gradients:

In order to calculate T_g , B_g , and C_g , we need to analyze how texture, color, and brightness vary across various shapes and sizes. Half-disc masks, which are binary picture pairs that represent half-disc shapes, are an effective way to achieve this. These masks are very helpful because, by filtering the data, we can compute chi-square (χ^2) distances much more quickly than if we were to iterate over each pixel neighborhood and aggregate histogram counts.

These masks are easy to create. Fig. 5 shows an example set of masks with three scales and eight orientations. With the use of these half-disc masks, distributions in opposite directions (such as left/right pairs) can be compared at the same scale, with a pixel at the center.

The degree of change in texture, brightness, and color distributions at each pixel is indicated by the T_g , B_g , and C_g values. By contrasting the distributions inside the left and right half-disc pairs, they are computed. The gradient is minor when the distributions are comparable and big when they differ greatly. The resulting local gradient measurements record the rate of change in texture, brightness, and color distributions across different scales and directions since the half-disc masks span many scales and orientations.

D. Pb-lite Output:

The last stage involves applying a straightforward equation to integrate data from the features with a baseline approach (based on Sobel or Canny edge detection, or an average of both).

$$PbEdges = \frac{(T_g + B_g + C_g)}{3} \odot (w_1 * cannyPb + w_2 * sobelPb)$$

Here, $\odot$ is the Hadamard product operator. The generated pb-lite boundaries for all the 10 provided images is shown in Figure.

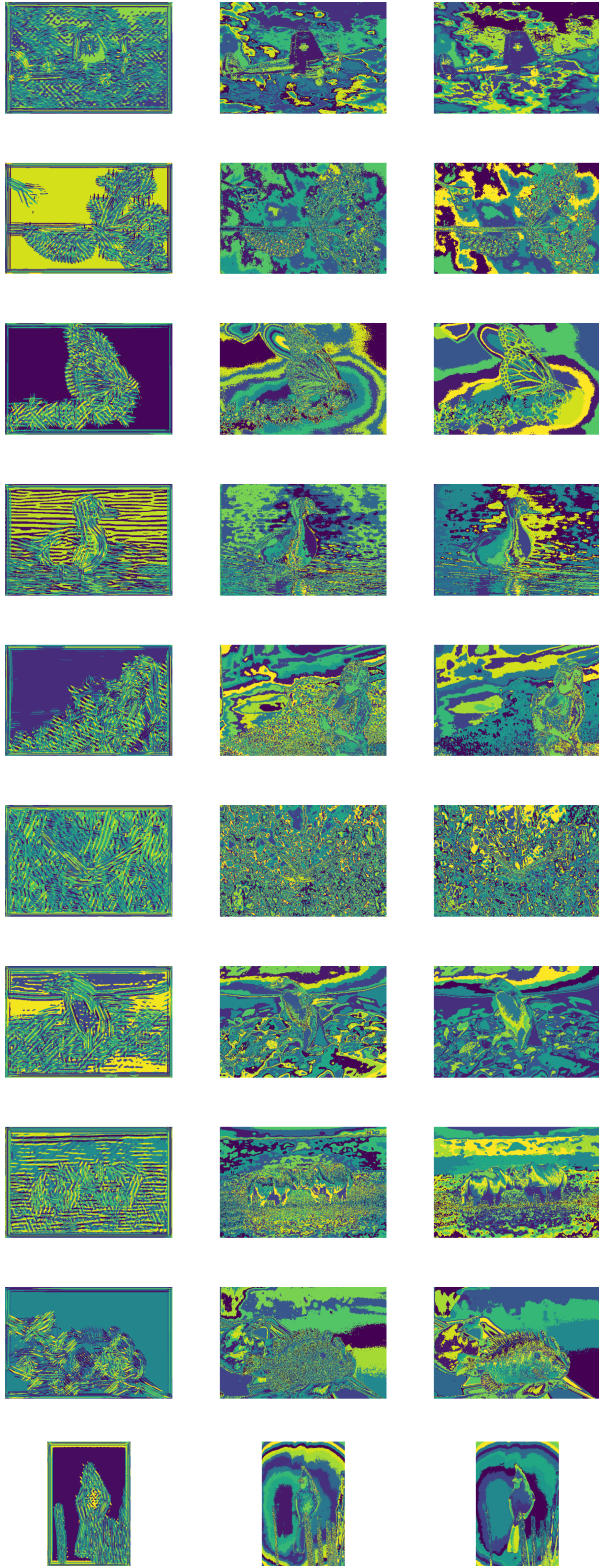


Fig. 3: Comparative visualization of Texton Maps (left), Brightness Maps (center), and Color Maps (right) for 10 images.

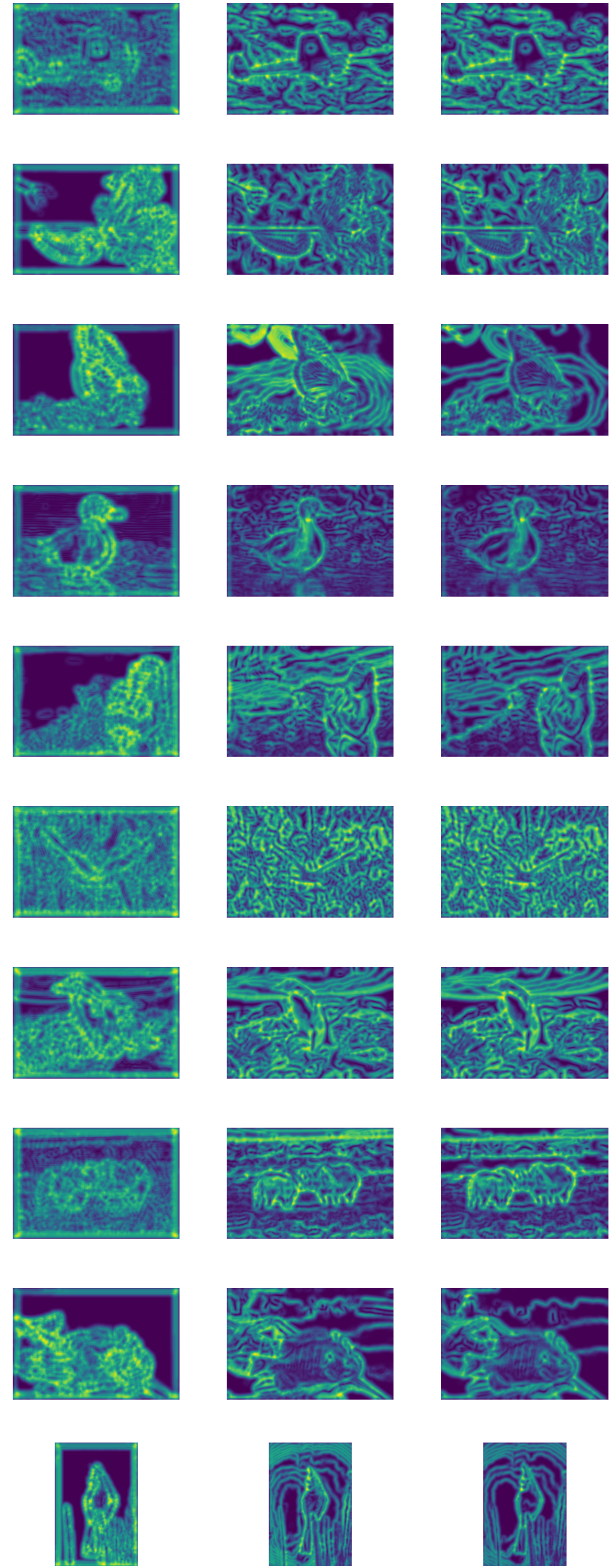


Fig. 4: Comparative visualization of Tg (left), Bg (center), and Cg (right) for 10 images.

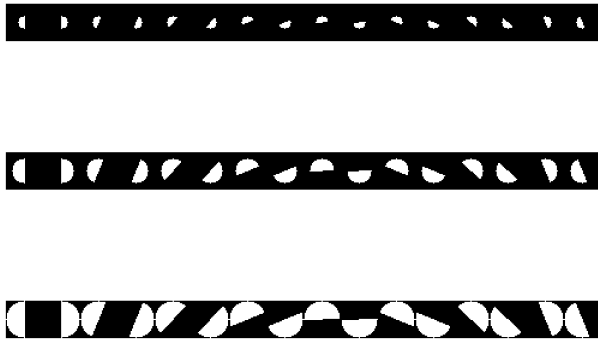


Fig. 5: Half Disk Filter.

E. Results and Analysis:

It is clear from the comparison that the Canny output has a lot of noise, whereas the Sobel output is the most suppressed. The Pb-lite result is the most balanced of the three techniques, and although thresholding has increased the effectiveness of boundary identification, it has also added some noise to the pictures.

II. PHASE 2: DEEP DIVE ON DEEP LEARNING

The homework's second phase focuses on deep learning for computer vision. We use a variety of neural networks to classify images in this area. The CIFAR-10 dataset, which includes 10,000 testing images and 50,000 training images, is used to train all networks.

Where possible, the majority of the hyperparameters are maintained constant to allow for an equitable comparison of the networks' performance. With the exception of the ResNeXt network, which is trained for 20 epochs, all networks are trained for 10 epochs due to time restrictions. With an initial learning rate of 0.001, the AdamW optimizer is used which is a modification of the Adam optimizer that separates weight decay from gradient update. It is frequently utilized because of its efficiency and effectiveness in training deep neural networks. A batch size of 32 is used for training.

A learning rate scheduler has been added to the Train.py file, which adjusts the learning rate during training according to a predetermined schedule. This can help improve the training process by slowing the learning rate as it proceeds. StepLR reduces the learning rate by gamma every step size epoch.

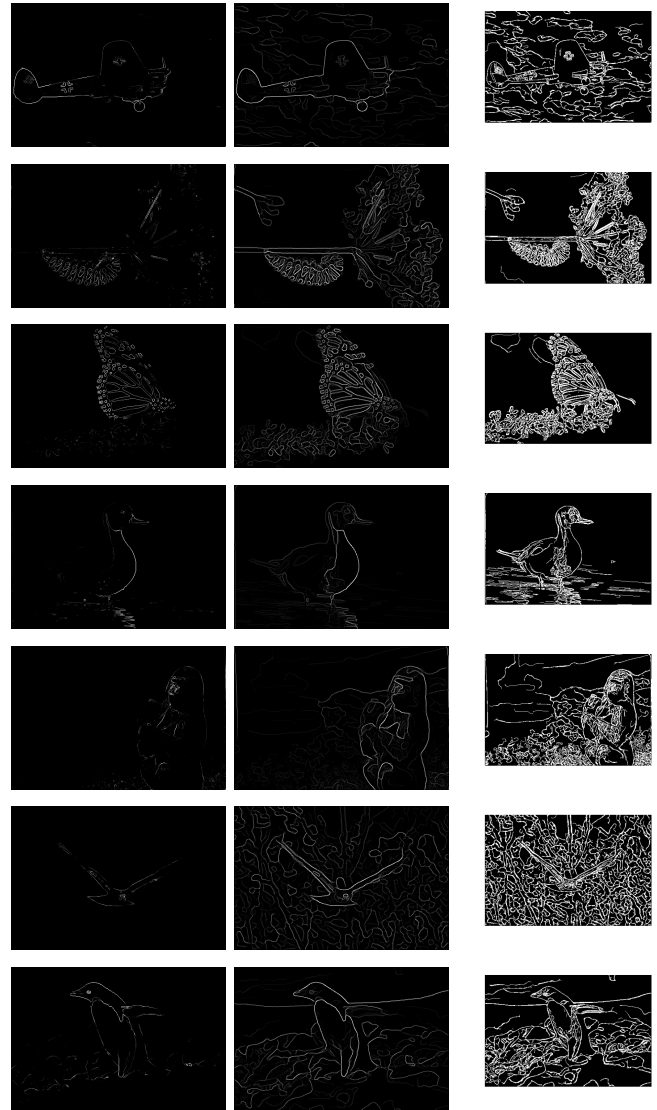
A. My First Neural Network:

The process starts with the first convolutional layer, written as conv1. The operations are performed in the following sequence:

Convolution: The input is passed through the conv1 layer, which uses convolution filters to extract low-level features such as edges and textures.

ReLU Activation: The Rectified Linear Unit (ReLU) activation function is used to incorporate nonlinearity into the model, allowing it to learn complex patterns.

Max Pooling: A max pooling procedure minimizes the spatial



dimensions of feature maps, increasing computing efficiency and focusing on the most important features.

For Basic CNN network with data augmentation has 1,147,466 trainable parameters. The training and testing accuracy and loss is shown in figure [8] and [9].

Following three convolutional layers, the feature maps are flattened into a one-dimensional tensor using, $x = x.view(-1, 128 * 4 * 4)$. This phase turns the 3D tensor (channels, height, and width) to a vector that may be used in fully connected layers. The flattened tensor is 128 by 4 by 4. After three 2x2 max-pooling procedures, the input picture was shrunk to 128x4x4. The first fully connected layer, fc1, processes the flattened tensor. The input features are mapped to a higher-dimensional space using learned weights. ReLU Activation introduces non-linearity, allowing the model to learn complicated correlations between features. The second fully connected layer (fc2) is the network's last layer.

I have refereed this CNN network architecture from Py-

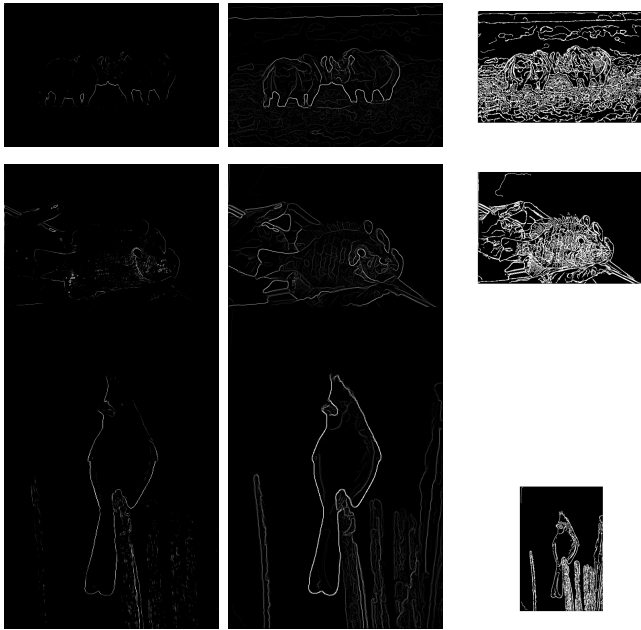


Fig. 7: Comparative visualization of Sobel (left), Canny (center), and Pb-lite (right) for 10 images.

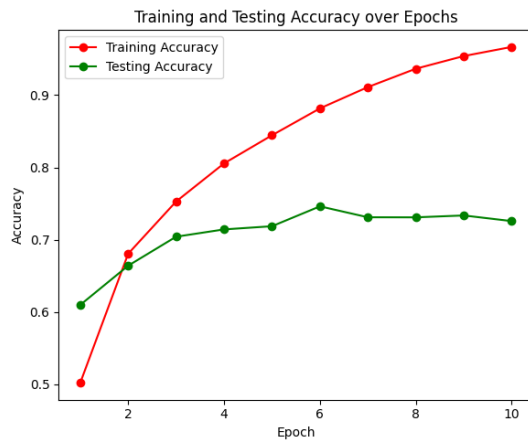


Fig. 8: Training and Testing Accuracy of Basic CNN Network.

Torch Neural network example webpage. This architecture is relatively simple and suitable for small datasets like CIFAR-10.

After traing basic CNN network, I have implemented custom CNN network by performing data augmentation and optimizer scheduler. Traininable parameters for this network are 1,147,866.

1) *Batch size reduction*: I have used these at start because of the computation limitation but later I came to know that batch size reduction are used to boost the model's performance and efficiency during validation. The loop executes 50 iterations, with the 'GenerateBatch' function creating a new batch of test data each time. This function accepts the test

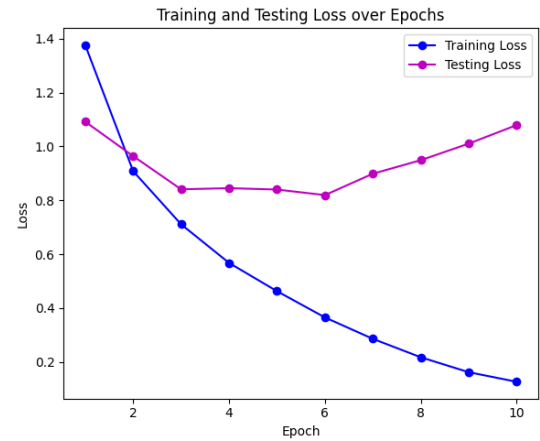


Fig. 9: Training and Testing Loss of Basic CNN Network.

```

Checkpoints/9model.ckpt Model Saved...
[4648 7 181 6 50 8 30 22 23 11] (0)
[ 14 4850 19 7 7 0 24 3 13 54] (1)
[ 30 0 4560 40 190 14 126 35 5 2] (2)
[ 5 6 100 4299 197 63 223 89 17 7] (3)
[ 9 0 47 19 4848 8 52 52 4 2] (4)
[ 0 1 120 404 226 4068 109 126 1 7] (5)
[ 0 0 29 14 38 3 4901 1 3 0] (6)
[ 3 0 23 11 83 6 5 4866 4 1] (7)
[125 22 32 7 18 3 29 8 4758 16] (8)
[ 41 90 15 14 17 5 17 25 13 4682] (9)
(0) (1) (2) (3) (4) (5) (6) (7) (8) (9)
Accuracy: 92.93026231605886 %

```

Fig. 10: Training Confusion Matrix of Basic CNN Network.

```

CheckPoint = torch.load(ModelPath)
Number of parameters in this model are 10
Testing time per image: 0.1920688723 ms
Number of parameters in this model are 10
[766 15 70 20 30 3 19 20 33 24] (0)
[ 21 849 12 5 6 3 18 9 16 61] (1)
[ 42 6 652 43 106 22 67 46 7 9] (2)
[ 18 9 84 491 119 91 108 58 9 13] (3)
[ 8 2 60 33 775 17 50 49 6 0] (4)
[ 6 3 77 194 77 498 53 75 9 8] (5)
[ 3 4 36 40 40 11 857 4 2 3] (6)
[ 8 1 30 31 76 29 12 808 0 5] (7)
[ 83 27 19 17 19 1 11 12 789 22] (8)
[ 31 94 16 13 9 3 14 32 20 768] (9)
(0) (1) (2) (3) (4) (5) (6) (7) (8) (9)
Accuracy: 72.53 %

```

Fig. 11: Testing Confusion Matrix of Basic CNN Network

dataset ('TestSet'), test labels ('TestLabels'), picture size ('ImageSize'), and a mini-batch size of 200 ('MiniBatchSize=200') as parameters. Setting the mini-batch size to 200 guarantees that each batch has 200 samples, which reduces computational overhead and memory utilization during validation.

2) *Data Augmentation*: Data augmentation and standardization is used here for training images to improve the model's generalization and robustness. It initializes 'I1Batch' and 'LabelBatch' to store augmented images and labels. A while loop runs until the specified 'MiniBatchSize' is reached, randomly selecting images and labels from the 'TrainSet'. Augmentation is performed using 'v2.Compose', including techniques like random cropping and resizing, horizontal flipping, type

conversion, and normalization. These transformations create variations in scale, position, and orientation, enhancing the model's ability to recognize diverse patterns. The augmented data is converted to tensors, moved to the target device, and returned, ensuring the model is trained on diverse examples for better performance on unseen data.

Optimizer and Hyperparameters for CNN Network

Optimizer: AdamW

Learning Rate: 0.001

Weight Decay: 1e-4

Batch Size

Batch Size: 32

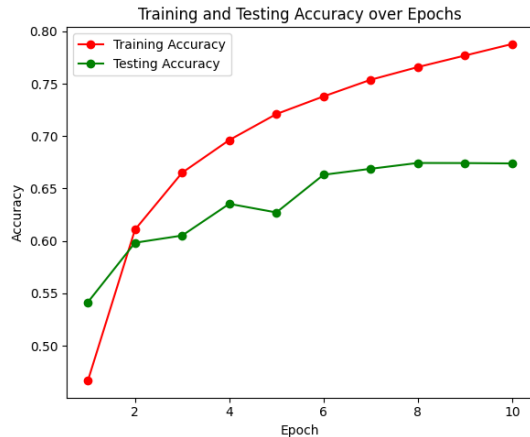


Fig. 12: Training and Testing Accuracy of CNN Network.

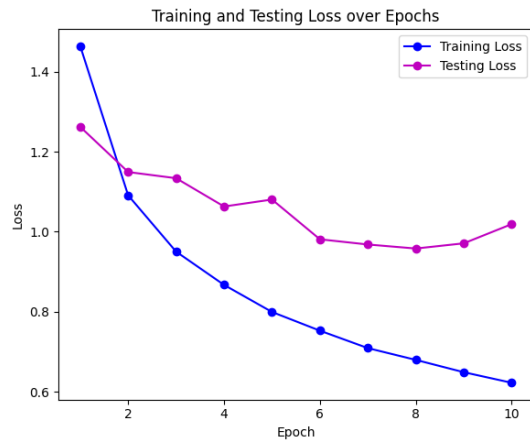


Fig. 13: Training and Testing Loss of CNN Network.

B. ResNet18 network:

ResNet (Residual Network) is a deep neural network design that uses residual (or skip) connections to address the vanishing gradient problem frequent in very deep networks. The key idea is to allow the network to learn residual functions relative to the input of each layer, hence allowing the training of deeper models.

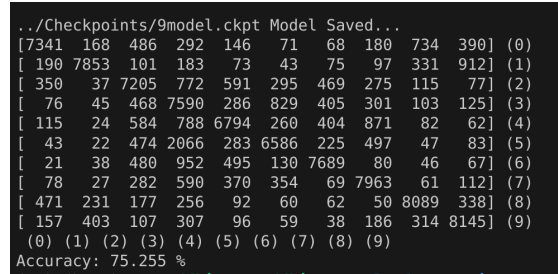


Fig. 14: Training Confusion Matrix of CNN Network.

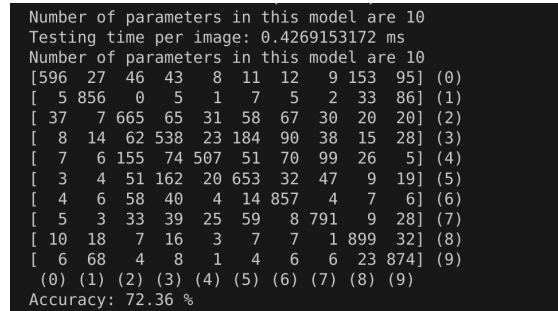


Fig. 15: Testing Confusion Matrix of CNN Network

Key Features:

BasicBlock: is ResNet's core unit.

ResNet Class: ResNet's fundamental structure, formed by stacking numerous BasicBlocks.

make layer Method: A utility function that builds layers out of BasicBlocks.

ResNet18 Function: A unique ResNet setup with 18 layers.

The ResNet implementation starts with an initial convolutional and pooling stage, which includes a 7x7 convolutional layer, batch normalization, ReLU activation, and max pooling to downsample the input. It then goes through four residual layers, each made up of stacked BasicBlocks with increasing filter sizes (64, 128, 256, 512) and strides for additional downsampling. An adaptive average pooling layer reduces spatial dimensions to 1x1, followed by a fully connected layer that converts the output into class scores. Each residual layer is built using the 'make layer' method, and the forward pass leads the input through these stages in order to process and classify the data quickly. For implementation of ResNet18, i have refered this website

Optimizer and Hyperparameters for ResNet18 Network

Optimizer: AdamW

Learning Rate: 0.001

Weight Decay: 1e-4

Batch Size

Batch Size: 32

For ResNet18, I got trainable parameters around 11,173,962.

C. ResNeXt50 network:

ResNeXt is an augmentation of the ResNet architecture that includes an additional dimension known as "cardinality" (the

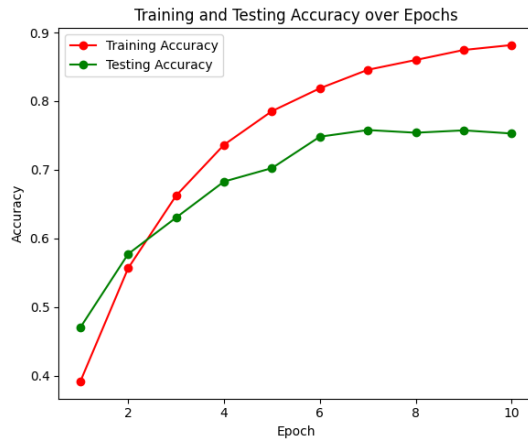


Fig. 16: Training and Testing Accuracy of ResNet18 Network.

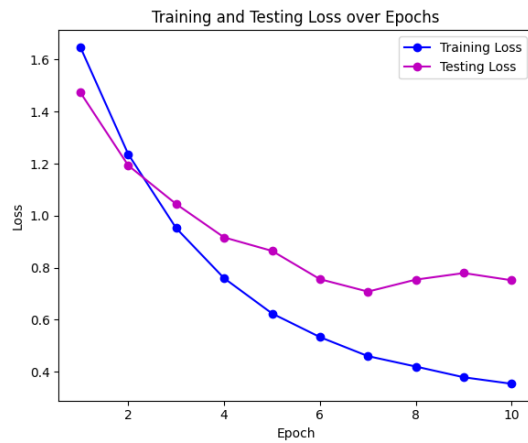


Fig. 17: Training and Testing Loss of ResNet18 Network.

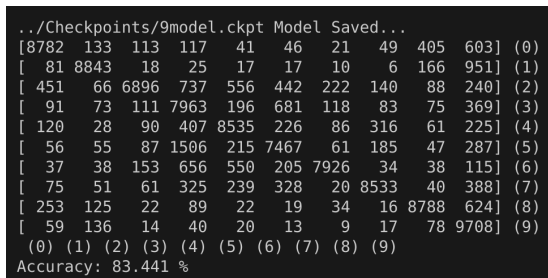


Fig. 18: Training Confusion Matrix of ResNet18 Network.

size of the collection of transformations). The main idea is to employ grouped convolutions to boost the model's capacity without considerably increasing computational complexity.

Key Components

ResNeXtBottleneck: The bottleneck block used in ResNeXt.

ResNeXt Class: The main ResNeXt architecture that stacks multiple ResNeXtBottleneck blocks.

make layer Method: A helper method to create layers of ResNeXtBottleneck blocks.

ResNeXt50 Function: A specific instance of ResNeXt with 50

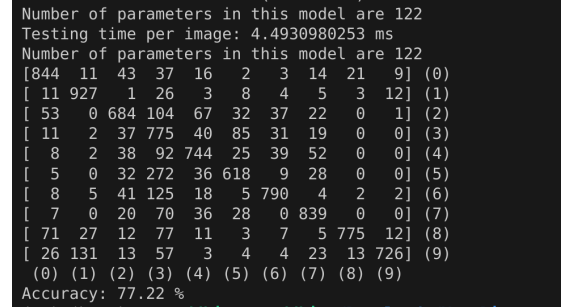


Fig. 19: Testing Confusion Matrix of ResNet18 Network

layers.

For ResNeXt50 network implementation, I referred to this webpage: ResNeXt Documentation. By changing hyperparameters and other values like learning rate and batch size, I was able to achieve a training accuracy of approximately 80 percent. For testing, I obtained an accuracy of around 68 percent. The number of trainable parameters in the model was 6,531,402.

Optimizer and Hyperparameters for ResNeXt50 Network

Optimizer: AdamW

Learning Rate: 0.001

Weight Decay: 1e-4

Batch Size

Batch Size: 48

Number of epochs: 20

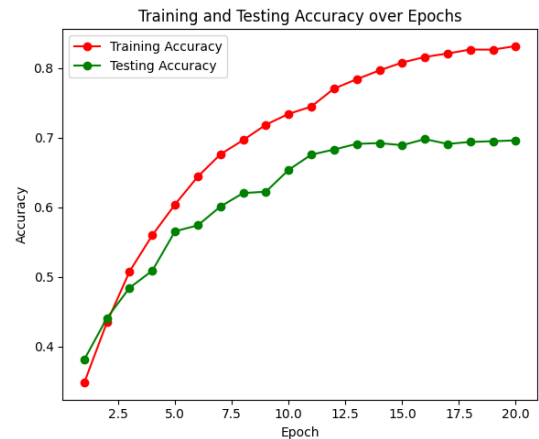


Fig. 20: Training and Testing Accuracy of ResNeXt50 Network.

D. DenseNet121 network:

DenseNet (Densely Connected Convolutional Networks) is a sort of convolutional neural network that connects each layer to the next layer in a feed-forward fashion. DenseNet uses all preceding layers' feature maps as inputs for each layer, as well as its own feature maps for all subsequent layers. This produces a dense connection pattern.

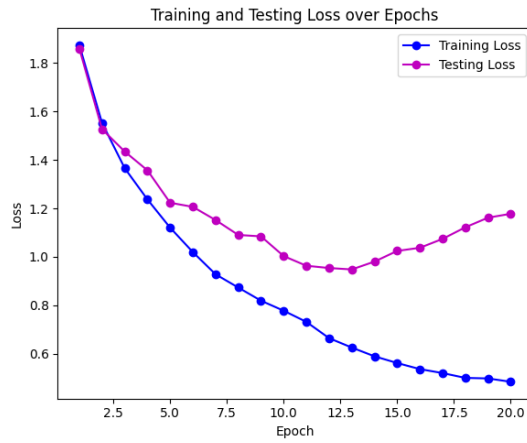


Fig. 21: Training and Testing Loss of ResNeXt50 Network.

```

../Checkpoints/19model.ckpt Model Saved...
[ 8331 223 263 100 57 56 55 60 602 191] (0)
[ 187 8886 64 65 32 44 37 38 256 539] (1)
[ 381 60 7803 293 363 278 418 157 121 72] (2)
[ 136 83 373 7249 238 965 492 219 144 147] (3)
[ 127 29 342 249 8074 202 426 473 117 73] (4)
[ 60 55 322 793 226 7746 170 350 82 126] (5)
[ 54 51 359 369 294 134 8648 33 59 53] (6)
[ 86 44 215 212 366 310 58 8312 80 167] (7)
[ 283 206 64 70 46 46 25 22 9137 213] (8)
[ 177 491 57 79 39 73 36 78 311 8555] (9)
(0) (1) (2) (3) (4) (5) (6) (7) (8) (9)
Accuracy: 82.71453134996801 %

```

Fig. 22: Training Confusion Matrix of ResNeXt50 Network.

```

Number of parameters in this model are 320
Testing time per image: 6.506100442 ms
Number of parameters in this model are 320
[ 703 16 89 34 16 6 7 6 106 17] (0)
[ 22 789 9 24 3 1 15 5 56 76] (1)
[ 70 9 551 96 106 43 74 22 20 9] (2)
[ 25 9 69 589 85 89 72 29 15 18] (3)
[ 32 4 113 92 601 20 67 55 12 4] (4)
[ 13 5 97 347 71 361 37 50 16 3] (5)
[ 9 10 52 118 79 11 701 5 5 10] (6)
[ 25 3 54 110 86 53 8 646 5 10] (7)
[ 74 25 13 37 6 2 6 0 815 22] (8)
[ 65 140 15 47 9 3 9 16 51 645] (9)
(0) (1) (2) (3) (4) (5) (6) (7) (8) (9)
Accuracy: 64.01 %

```

Fig. 23: Testing Confusion Matrix of ResNeXt50 Network

Number of epochs: 10

For DenseNet121, I received trainable parameters as 1,000,618.

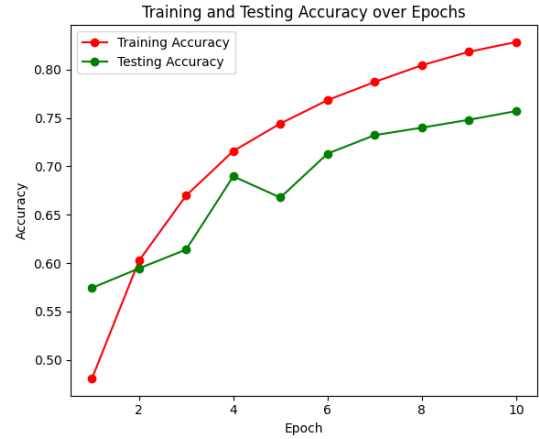


Fig. 24: Training and Testing Accuracy of DenseNet121 Network.

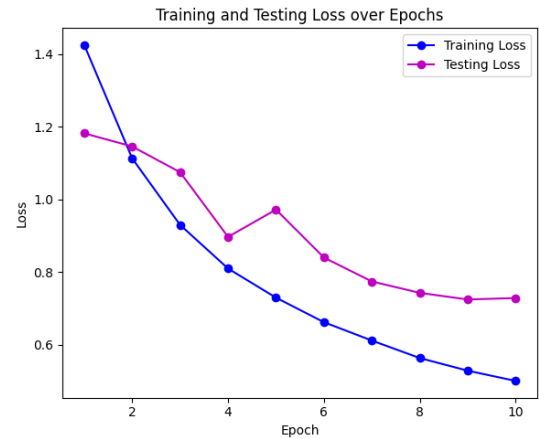


Fig. 25: Training and Testing Loss of DenseNet121 Network.

Key Components:

DenseLayer: The fundamental building unit of DenseNet.

DenseBlock: A block made up of numerous DenseLayers.

TransitionLayer: A layer that reduces the spatial dimension and number of feature maps between DenseBlocks.

DenseNet Class: The primary DenseNet architecture that combines numerous DenseBlocks and TransitionLayers.

DenseNet121 Function: An instance of DenseNet with 121 layers.

Optimizer and Hyperparameters for ResNeXt50 Network

Optimizer: AdamW

Learning Rate: 0.001

Weight Decay: 1e-4

Batch Size

Batch Size: 32

```

../Checkpoints/9model.ckpt Model Saved...
[ 8200 68 142 164 78 48 34 160 833 143] (0)
[ 160 8380 24 126 31 31 34 70 558 326] (1)
[ 649 13 6921 726 463 311 370 337 224 40] (2)
[ 146 32 190 7466 277 1065 274 404 175 69] (3)
[ 94 6 232 500 7674 243 266 859 125 15] (4)
[ 63 22 144 1592 239 6984 152 614 98 40] (5)
[ 123 14 231 744 322 138 8097 125 83 21] (6)
[ 88 12 128 370 204 258 32 9018 81 49] (7)
[ 293 52 26 91 44 25 20 37 9398 98] (8)
[ 208 367 34 231 32 37 30 165 570 8412] (9)
(0) (1) (2) (3) (4) (5) (6) (7) (8) (9)
Accuracy: 80.52423224568138 %

```

Fig. 26: Training Confusion Matrix of DenseNet121 Network.

E. Results and Analysis:

Each model comes with its own advantages and limitations. The selection of a model depends on the specific needs of the task, including dataset complexity, computational resources,


```

Number of parameters in this model are 722
Testing time per image: 13.3437078658 ms
Number of parameters in this model are 722
[766 27 39 5 38 0 11 3 103 8] (0)
[ 6 961 2 0 1 0 3 0 22 5] (1)
[ 53 1 630 51 100 26 104 10 25 0] (2)
[ 32 14 41 620 60 90 89 18 35 1] (3)
[ 9 7 30 43 768 19 96 20 8 0] (4)
[ 15 10 43 211 71 571 45 26 8 0] (5)
[ 10 5 29 30 15 5 890 1 15 0] (6)
[ 20 15 30 35 131 22 12 722 11 2] (7)
[ 39 14 4 9 5 0 5 0 923 1] (8)
[ 28 195 3 17 1 0 12 2 38 704] (9)
(0) (1) (2) (3) (4) (5) (6) (7) (8) (9)
Accuracy: 75.55 %

```

Fig. 27: Testing Confusion Matrix of DenseNet121 Network

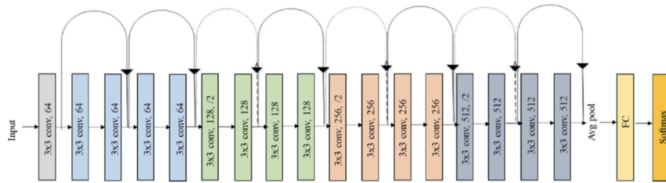


Fig. 28: ResNet18 Architecture



Fig. 29: ResNeXt Architecture

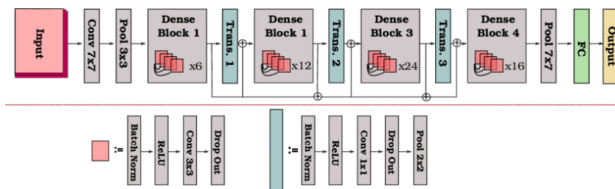


Fig. 30: DenseNet121 Architecture

Network	Training Accuracy (%)	Testing Accuracy (%)	Trainable parameters	Testing time per image(jms)
Basic CNN	92.93	72.57	1,147,466	0.19
Custom CNN	75.25	72.36	1,147,866	0.42
ResNet18	83.44	72.22	11,173,962	4.49
ResNeXt50	82.71	64.01	6,531,402	6.5
DenseNet121	80.52	75.55	1,000,618	13.34

Fig. 31: Table summarizing and comparing all models

REFERENCES

- [1] Team, T. A. (2021, January 1). ResNeXt: from scratch. Towards AI. <https://towardsai.net/p/research/resnext-from-scratch>.
- [2] Arora, A. (2020, August 2). DenseNet Architecture Explained with PyTorch Implementation from TorchVision. <https://amaarora.github.io/posts/2020-08-02-densenets.html>
- [3] Writing ResNet from Scratch in PyTorch — DigitalOcean. (n.d.). <https://www.digitalocean.com/community/tutorials/writing-resnet-from-scratch-in-pytorch>

and performance goals. For quick testing or smaller datasets, the Custom CIFAR10Model is often adequate. However, for more challenging tasks and higher performance, models like ResNet18, ResNeXt50, and DenseNet121 are better suited. Among these, DenseNet121 generally delivers the best performance, at the cost of higher demands on training time and memory.

The detailed comparison between all five network, with their training accuracy, testing accuracy, trainable parameters and testing time required for each image is given in the table: